

Principles of Programming Languages

Lecture 4: Side-effects. I/O. Local variables.

Andrei Arusoaie¹

¹Department of Computer Science

October 25, 2016

Outline

Side-effects

Input/Output

Local variables

Outline

Side-effects

Input/Output

Local variables

Outline

Side-effects

Input/Output

Local variables

Side-effects

- ▶ What are side-effects in computer programming?
- ▶ **observable** change over the *“outside world”*
- ▶ Example: expression $2 + 3$
 - ▶ **evaluates** to 5
- ▶ Example: expression $++x$
 - ▶ **evaluates** to $x + 1$ **and** **changes** the state

Side-effects

- ▶ What are side-effects in computer programming?
- ▶ **observable** change over the “*outside world*”
- ▶ Example: expression $2 + 3$
 - ▶ **evaluates** to 5
- ▶ Example: expression $++x$
 - ▶ **evaluates** to $x + 1$ **and** **changes** the state

Side-effects

- ▶ What are side-effects in computer programming?
- ▶ **observable** change over the “*outside world*”
- ▶ Example: expression $2 + 3$
 - ▶ evaluates to 5
- ▶ Example: expression $++x$
 - ▶ evaluates to $x + 1$ and changes the state

Side-effects

- ▶ What are side-effects in computer programming?
- ▶ **observable** change over the “*outside world*”
- ▶ Example: expression $2 + 3$
 - ▶ **evaluates** to 5
- ▶ Example: expression $++x$
 - ▶ **evaluates** to $x + 1$ **and** **changes** the state

Side-effects

- ▶ What are side-effects in computer programming?
- ▶ **observable** change over the “*outside world*”
- ▶ Example: expression $2 + 3$
 - ▶ **evaluates** to 5
- ▶ Example: expression $++x$
 - ▶ **evaluates** to $x + 1$ **and** **changes** the state

Side-effects

- ▶ What are side-effects in computer programming?
- ▶ **observable** change over the “*outside world*”
- ▶ Example: expression $2 + 3$
 - ▶ **evaluates** to 5
- ▶ Example: expression $++x$
 - ▶ **evaluates** to $x + 1$ **and** **changes** the state

Increment in K

- ▶ Syntax:

```
syntax AExp ::= "++" Id ";"
```

- ▶ Semantics:

```
rule <k> ++X => I +Int 1 ...</k>  
  <env>... X |-> (I => I +Int 1) ...</env>
```

- ▶ Demo

Increment in K

- ▶ Syntax:

`syntax AExp ::= "++" Id ";"`

- ▶ Semantics:

`rule <k> ++X => I +Int 1 ...</k>
 <env>... X |-> (I => I +Int 1) ...</env>`

- ▶ Demo

Increment in K

- ▶ Syntax:

`syntax AExp ::= "++" Id ";"`

- ▶ Semantics:

`rule <k> ++X => I +Int 1 ...</k>
 <env>... X |-> (I => I +Int 1) ...</env>`

- ▶ Demo

Input/Output

- ▶ **Semantics of I/O**
- ▶ Input: read an integer
 - ▶ `read()`
- ▶ Output: print an expression
 - ▶ `print(x);`

Input/Output

- ▶ Semantics of I/O
- ▶ Input: read an integer
 - ▶ `read()`
- ▶ Output: print an expression
 - ▶ `print(x);`

Input/Output

- ▶ Semantics of I/O
- ▶ Input: read an integer
 - ▶ `read()`
- ▶ Output: print an expression
 - ▶ `print(x);`

Input

Steps:

1. Add syntax declaration:

```
syntax AExp ::= "read" "(" " ")"
```

2. Add a new cell connected to stdin:

```
<in stream="stdin"> .List </in>
```

3. Add a rule for reading a value:

```
rule <k> (read() => I) ...</k>
```

```
<in> (ListItem(I:Int) => .List) ...</in>
```

DEMO

Input

Steps:

1. Add syntax declaration:

```
syntax AExp ::= "read" "(" " " "
```

2. Add a new cell connected to stdin:

```
<in stream="stdin"> .List </in>
```

3. Add a rule for reading a value:

```
rule <k> (read() => I) ...</k>
```

```
<in> (ListItem(I:Int) => .List) ...</in>
```

DEMO

Input

Steps:

1. Add syntax declaration:

```
syntax AExp ::= "read" "(" " " "
```

2. Add a new cell connected to stdin:

```
<in stream="stdin"> .List </in>
```

3. Add a rule for reading a value:

```
rule <k> (read() => I) ...</k>
```

```
<in> (ListItem(I:Int) => .List) ...</in>
```

DEMO

Input

Steps:

1. Add syntax declaration:

```
syntax AExp ::= "read" "(" " " "
```

2. Add a new cell connected to stdin:

```
<in stream="stdin"> .List </in>
```

3. Add a rule for reading a value:

```
rule <k> (read() => I) ...</k>
```

```
<in> (ListItem(I:Int) => .List) ...</in>
```

DEMO

Input

Steps:

1. Add syntax declaration:

```
syntax AExp ::= "read" "(" " " "
```

2. Add a new cell connected to stdin:

```
<in stream="stdin"> .List </in>
```

3. Add a rule for reading a value:

```
rule <k> (read() => I) ...</k>
```

```
<in> (ListItem(I:Int) => .List) ...</in>
```

DEMO

Output

Steps:

1. Add syntax declaration:

```
syntax Stmt ::= "print" "(" AExp ")"
```

2. Add a new cell connected to stdout:

```
<out stream="stdout"> .List </out>
```

3. Add a rule that outputs a value:

```
rule <k> (print(I:Int); => .) ...</k>  
<out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Steps:

1. Add syntax declaration:

```
syntax Stmt ::= "print" "(" AExp ")"
```

2. Add a new cell connected to stdout:

```
<out stream="stdout"> .List </out>
```

3. Add a rule that outputs a value:

```
rule <k> (print(I:Int); => .) ...</k>  
<out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Steps:

1. Add syntax declaration:

```
syntax Stmt ::= "print" "(" AExp ")"
```

2. Add a new cell connected to stdout:

```
<out stream="stdout"> .List </out>
```

3. Add a rule that outputs a value:

```
rule <k> (print(I:Int); => .) ...</k>  
<out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Steps:

1. Add syntax declaration:

```
syntax Stmt ::= "print" "(" AExp ")"
```

2. Add a new cell connected to stdout:

```
<out stream="stdout"> .List </out>
```

3. Add a rule that outputs a value:

```
rule <k> (print(I:Int); => .) ...</k>  
<out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Steps:

1. Add syntax declaration:

```
syntax Stmt ::= "print" "(" AExp ")"
```

2. Add a new cell connected to stdout:

```
<out stream="stdout"> .List </out>
```

3. Add a rule that outputs a value:

```
rule <k> (print(I:Int); => .) ...</k>  
  <out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Print strings:

1. Add syntax declaration:

```
syntax AExp ::= String
```

2. Add a new **sort** to group the printable expressions:

```
syntax Printable ::= Int | String  
syntax AExp ::= Printable
```

3. Modify the output rule:

```
rule <k> (print(I:Printable); => .) ...</k>  
<out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Print strings:

1. Add syntax declaration:

```
syntax AExp ::= String
```

2. Add a new **sort** to group the printable expressions:

```
syntax Printable ::= Int | String  
syntax AExp ::= Printable
```

3. Modify the output rule:

```
rule <k> (print(I:Printable); => .) ...</k>  
<out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Print strings:

1. Add syntax declaration:

```
syntax AExp ::= String
```

2. Add a new **sort** to group the printable expressions:

```
syntax Printable ::= Int | String  
syntax AExp ::= Printable
```

3. Modify the output rule:

```
rule <k> (print(I:Printable); => .) ...</k>  
      <out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Print strings:

1. Add syntax declaration:

```
syntax AExp ::= String
```

2. Add a new **sort** to group the printable expressions:

```
syntax Printable ::= Int | String  
syntax AExp ::= Printable
```

3. Modify the output rule:

```
rule <k> (print(I:Printable); => .) ...</k>  
      <out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Print strings:

1. Add syntax declaration:

```
syntax AExp ::= String
```

2. Add a new **sort** to group the printable expressions:

```
syntax Printable ::= Int | String  
syntax AExp ::= Printable
```

3. Modify the output rule:

```
rule <k> (print(I:Printable); => .) ...</k>  
    <out>... (.List => ListItem(I)) </out>
```

DEMO

Output

Print strings:

1. Add syntax declaration:

```
syntax AExp ::= String
```

2. Add a new **sort** to group the printable expressions:

```
syntax Printable ::= Int | String  
syntax AExp ::= Printable
```

3. Modify the output rule:

```
rule <k> (print(I:Printable); => .) ...</k>  
    <out>... (.List => ListItem(I)) </out>
```

DEMO

Local variables

- ▶ How to handle local variables in blocks?

```
▶ int x, y;  
  x = 1;  
  y = 10;  
  {  
    int x;  
    x = 3;  
    print(x); print(y);  
  }  
  print(x); print(y);
```

- ▶ **Visibility, Hidding**

Local variables

- ▶ How to handle local variables in blocks?

```
▶ int x, y;  
  x = 1;  
  y = 10;  
  {  
    int x;  
    x = 3;  
    print(x); print(y);  
  }  
print(x); print(y);
```

- ▶ Visibility, Hidding

Local variables

- ▶ How to handle local variables in blocks?

```
▶ int x, y;  
  x = 1;  
  y = 10;  
  {  
    int x;  
    x = 3;  
    print(x); print(y);  
  }  
  print(x); print(y);
```

- ▶ **Visibility, Hidding**

Local variables: behavior

- ▶ When entering the block:
 - ▶ Redeclared variables **hide** previous declarations (e.g., x)
 - ▶ The undeclared (inherited) variables are made **visible** in the block (e.g., y)
- ▶ When exiting the block:
 - ▶ Redeclared variables are de-activated (eventually de-allocated)
 - ▶ The old environment is restored
- ▶ Solutions?

Local variables: behavior

- ▶ When entering the block:
 - ▶ Redeclared variables **hide** previous declarations (e.g., x)
 - ▶ The undeclared (inherited) variables are made **visible** in the block (e.g., y)
- ▶ When exiting the block:
 - ▶ Redeclared variables are de-activated (eventually de-allocated)
 - ▶ The old environment is restored
- ▶ Solutions?

Local variables: behavior

- ▶ When entering the block:
 - ▶ Redeclared variables **hide** previous declarations (e.g., x)
 - ▶ The undeclared (inherited) variables are made **visible** in the block (e.g., y)
- ▶ When exiting the block:
 - ▶ Redeclared variables are de-activated (eventually de-allocated)
 - ▶ The old environment is restored
- ▶ Solutions?

Local variables: behavior

- ▶ When entering the block:
 - ▶ Redeclared variables **hide** previous declarations (e.g., x)
 - ▶ The undeclared (inherited) variables are made **visible** in the block (e.g., y)
- ▶ When exiting the block:
 - ▶ Redeclared variables are de-activated (eventually de-allocated)
 - ▶ The old environment is restored
- ▶ Solutions?

Local variables: behavior

- ▶ When entering the block:
 - ▶ Redeclared variables **hide** previous declarations (e.g., x)
 - ▶ The undeclared (inherited) variables are made **visible** in the block (e.g., y)
- ▶ When exiting the block:
 - ▶ Redeclared variables are de-activated (eventually de-allocated)
 - ▶ The old environment is restored
- ▶ Solutions?

Local variables: behavior

- ▶ When entering the block:
 - ▶ Redeclared variables **hide** previous declarations (e.g., x)
 - ▶ The undeclared (inherited) variables are made **visible** in the block (e.g., y)
- ▶ When exiting the block:
 - ▶ Redeclared variables are de-activated (eventually de-allocated)
 - ▶ The old environment is restored
- ▶ Solutions?

Local variables: behavior

- ▶ When entering the block:
 - ▶ Redeclared variables **hide** previous declarations (e.g., x)
 - ▶ The undeclared (inherited) variables are made **visible** in the block (e.g., y)
- ▶ When exiting the block:
 - ▶ Redeclared variables are de-activated (eventually de-allocated)
 - ▶ The old environment is restored
- ▶ Solutions?

Local variables in K

Memory refinement:

- ▶ **environment** + **variable references**
- ▶ $\langle \text{env} \rangle : \text{varname} \mapsto \text{reference}$
- ▶ $\langle \text{store} \rangle : \text{reference} \mapsto \text{value}$

Solution: keep the environment organised in a stack

1. either in the k cell
2. or use a dedicated $\langle \text{stack} \rangle$

Local variables in K

Memory refinement:

- ▶ **environment** + **variable references**
- ▶ $\langle \text{env} \rangle : \text{varname} \mapsto \text{reference}$
- ▶ $\langle \text{store} \rangle : \text{reference} \mapsto \text{value}$

Solution: keep the environment organised in a stack

1. either in the k cell
2. or use a dedicated $\langle \text{stack} \rangle$

Local variables in K

Memory refinement:

- ▶ **environment** + **variable references**
- ▶ `<env>: varname  $\mapsto$  reference`
- ▶ `<store>: reference  $\mapsto$  value`

Solution: keep the environment organised in a stack

1. either in the `k` cell
2. or use a dedicated `<stack>`

Local variables in K

Memory refinement:

- ▶ **environment** + **variable references**
- ▶ `<env>: varname  $\mapsto$  reference`
- ▶ `<store>: reference  $\mapsto$  value`

Solution: keep the environment organised in a stack

1. either in the `k` cell
2. or use a dedicated `<stack>`

Solution 1: stack in the $\langle k \rangle$ cell

Steps:

1. memory refinement + update rules: assignment, lookup
2. Revise declarations rule:

```
rule  $\langle k \rangle$  int (X, Xs  $\Rightarrow$  Xs); ... $\langle /k \rangle$   
   $\langle \text{env} \rangle$  Rho:Map  $\Rightarrow$  Rho[X  $\leftarrow$  !L:Int]  $\langle / \text{env} \rangle$   
   $\langle \text{store} \rangle$  Sigma:Map (.  $\Rightarrow$  (!L  $\mid \rightarrow$  0))  $\langle / \text{store} \rangle$ 
```

3. Replace the rule for non-empty block with:

```
rule  $\langle k \rangle$  { S:Stmt }  $\Rightarrow$  S  $\curvearrowright$  Rho ... $\langle /k \rangle$   $\langle \text{env} \rangle$  Rho  $\langle / \text{env} \rangle$   
rule  $\langle k \rangle$  Rho  $\Rightarrow$  . ... $\langle /k \rangle$   $\langle \text{env} \rangle$  _  $\Rightarrow$  Rho  $\langle / \text{env} \rangle$ 
```

Solution 1: stack in the $\langle k \rangle$ cell

Steps:

1. memory refinement + update rules: assignment, lookup
2. Revise declarations rule:

```
rule  $\langle k \rangle$  int (X, Xs  $\Rightarrow$  Xs); ... $\langle k \rangle$   
   $\langle \text{env} \rangle$   $\text{Rho}:\text{Map} \Rightarrow \text{Rho}[X \leftarrow !L:\text{Int}]$   $\langle / \text{env} \rangle$   
   $\langle \text{store} \rangle$   $\text{Sigma}:\text{Map} ( . \Rightarrow (!L \mapsto 0))$   $\langle / \text{store} \rangle$ 
```

3. Replace the rule for non-empty block with:

```
rule  $\langle k \rangle$  { S:Stmt }  $\Rightarrow S \curvearrowright \text{Rho}$  ... $\langle k \rangle$   $\langle \text{env} \rangle$   $\text{Rho}$   $\langle / \text{env} \rangle$   
rule  $\langle k \rangle$   $\text{Rho} \Rightarrow .$  ... $\langle k \rangle$   $\langle \text{env} \rangle$   $\_ \Rightarrow \text{Rho}$   $\langle / \text{env} \rangle$ 
```


Solution 1: stack in the $\langle k \rangle$ cell

Steps:

1. memory refinement + update rules: assignment, lookup
2. Revise declarations rule:

```
rule  $\langle k \rangle$  int (X, Xs  $\Rightarrow$  Xs); ... $\langle k \rangle$   
   $\langle \text{env} \rangle$   $\text{Rho}:\text{Map} \Rightarrow \text{Rho}[X \leftarrow !L:\text{Int}]$   $\langle / \text{env} \rangle$   
   $\langle \text{store} \rangle$   $\text{Sigma}:\text{Map} ( . \Rightarrow (!L \mapsto 0))$   $\langle / \text{store} \rangle$ 
```

3. Replace the rule for non-empty block with:

```
rule  $\langle k \rangle$  { S:Stmt }  $\Rightarrow S \curvearrowright \text{Rho} \dots \langle k \rangle$   $\langle \text{env} \rangle$   $\text{Rho}$   $\langle / \text{env} \rangle$   
rule  $\langle k \rangle$   $\text{Rho} \Rightarrow . \dots \langle k \rangle$   $\langle \text{env} \rangle$   $\_ \Rightarrow \text{Rho}$   $\langle / \text{env} \rangle$ 
```

Solution 2: stack in dedicated cell

Steps:

1. Append a new cell: `<stack> .List </stack>`
2. Restore point: `syntax KItem ::= "restoreEnv"`
3. Replace the rule for non-empty block with:

```
rule <k> { S:Stmt } => S  $\curvearrowright$  restoreEnv ...</k>
  <env> Rho </env>
  <stack> (. => ListItem(Rho)) ...</stack>

rule <k> restoreEnv => . ...</k>
  <env> _ => Rho </env>
  <stack> (ListItem(Rho) => .List) ...</stack>
```

Solution 2: stack in dedicated cell

Steps:

1. Append a new cell: `<stack> .List </stack>`
2. Restore point: `syntax KItem ::= "restoreEnv"`
3. Replace the rule for non-empty block with:

```
rule <k> { S:Stmt } => S  $\curvearrowright$  restoreEnv ...</k>
  <env> Rho </env>
  <stack> (. => ListItem(Rho)) ...</stack>

rule <k> restoreEnv => . ...</k>
  <env> _ => Rho </env>
  <stack> (ListItem(Rho) => .List) ...</stack>
```

Solution 2: stack in dedicated cell

Steps:

1. Append a new cell: `<stack> .List </stack>`
2. Restore point: `syntax KItem ::= "restoreEnv"`
3. Replace the rule for non-empty block with:

```
rule <k> { S:Stmt } => S ↪ restoreEnv ...</k>
  <env> Rho </env>
  <stack> (. => ListItem(Rho)) ...</stack>

rule <k> restoreEnv => . ...</k>
  <env> _ => Rho </env>
  <stack> (ListItem(Rho) => .List) ...</stack>
```

Lab this week

- ▶ Given some language specification, you will create an interpreter for that language using K